Miko-专注力机制AI安全测试 Agent [开源]

Miko：开源AI安全探路者

Miko 是一个运行在本机的桌面应用——基于 Tauri 2 + React 构建，内置多款安全工具和skills，开箱即用，适配国内API（文中示例使用deepseek）。它的定位是渗透测试中的"AI 探路者"：不是黑盒替测试工程师做决策，而是在信息收集和探测阶段持续深挖、记住每一步发现、把探索路径清晰地摊在用户面前。

如果你用过通用 AI 编程助手做安全测试，大概都遇到过同一个问题：问一句答一句，没有记忆。第二轮对话不知道第一轮扫出了什么、第三轮忘了第二轮的线索——Agent 像金鱼一样每次从头开始。更麻烦的是，会话越积越多，散落在一堆标签页里，每次都要手动整理探测记录、拼测试文档，隔天打开还得先翻半天回忆上次探到哪了。Miko 要解决的正是这一整套问题。

黑板机制：让 Agent 记住自己探过什么

Miko 的核心设计基于一个前沿的 Agent 架构思路：将调度编排与执行探索分层，由 LLM 直接承担调度决策，形成"会话主 Agent 自己跑 OODA 循环"的运转模式。这个方向在近两年的多 Agent 协作和自主安全测试研究中已有论文探讨——核心思想是把"下一步探什么"的决策权留在上下文最完整的调度层，执行层只接收限定任务、干完汇报、不参与决策。

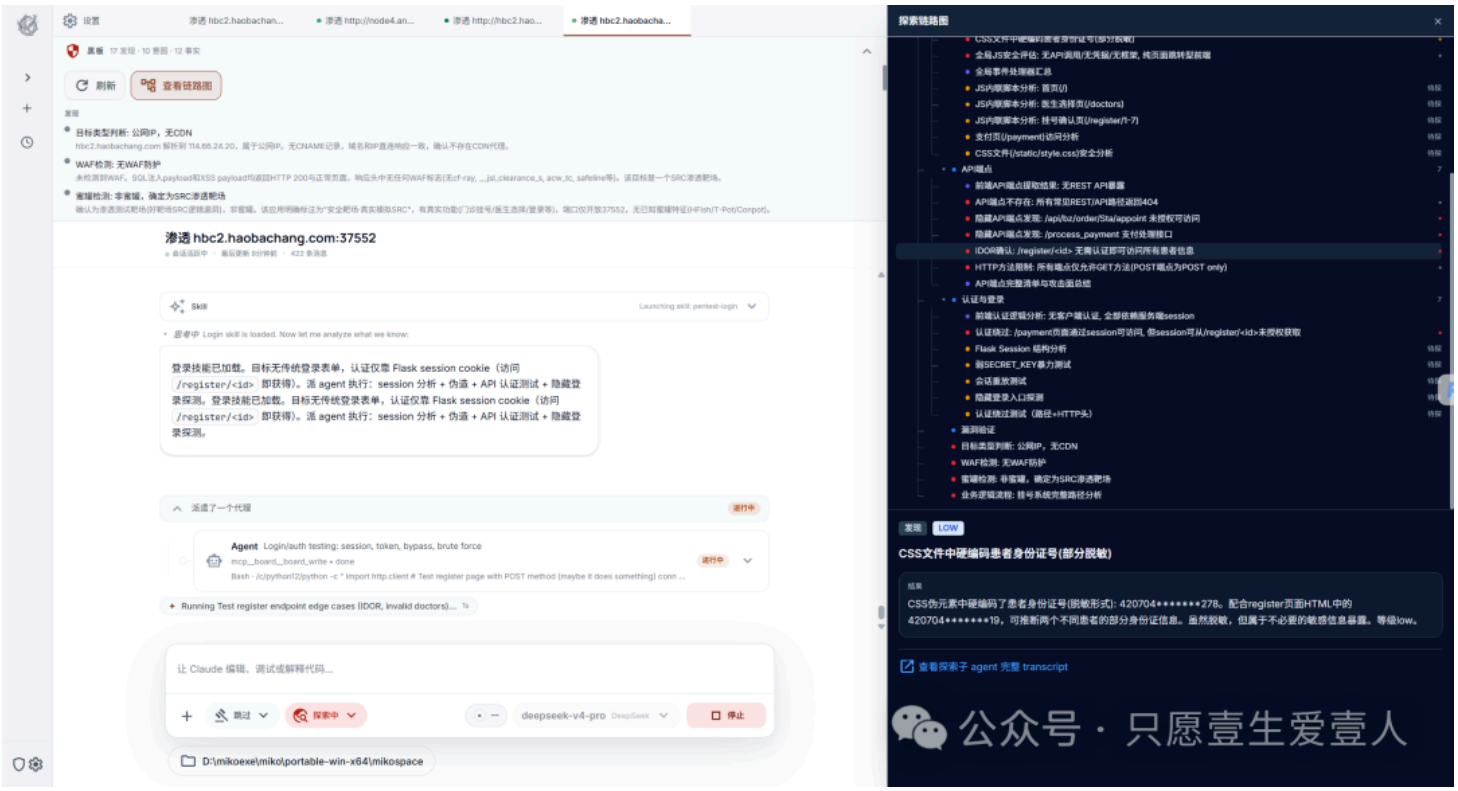
具体来说，用户开启安全测试模式后，会话主 Agent 扮演 dispatcher 角色——它持有一块"黑板"（共享作战地图），上面记录着三类信息：

- **Facts** ：已确认的客观事实。比如"端口 8888 开放""技术栈 PHP 8.1""`/.git` 目录存在泄露"。
- **Intents** ：想探但还没探完的方向。比如"SSRF 待测""登录绕过待试"。
- **Findings** ：确认或疑似的安全发现。比如"精度舍入漏洞""SQL 注入"。

主 Agent 的运作逻辑是：读黑板 → 对照目标判断哪些入口还没覆盖 → 挑一个待探 intent → 派探索子 agent 去打 → 解析子 agent 返回的结果 → 写回黑板 → 循环。这个循环持续到目标达成、覆盖度足够，或者用户主动介入纠偏。

探索子 agent 本身不碰黑板——它只接收 dispatcher 喂给它的一张"图快照"（当前已知的事实和待探方向），带着明确任务去打目标，然后把结果以结构化 JSON 吐回主 agent。

黑板上的每一个节点——无论是 asset（攻击面对象，如子域名、端口、接口）、fact、intent 还是 finding——都带 `parentId`，连成一棵深度不固定的探索树，用户可以折叠、展开、点击任意节点看三槽细节。

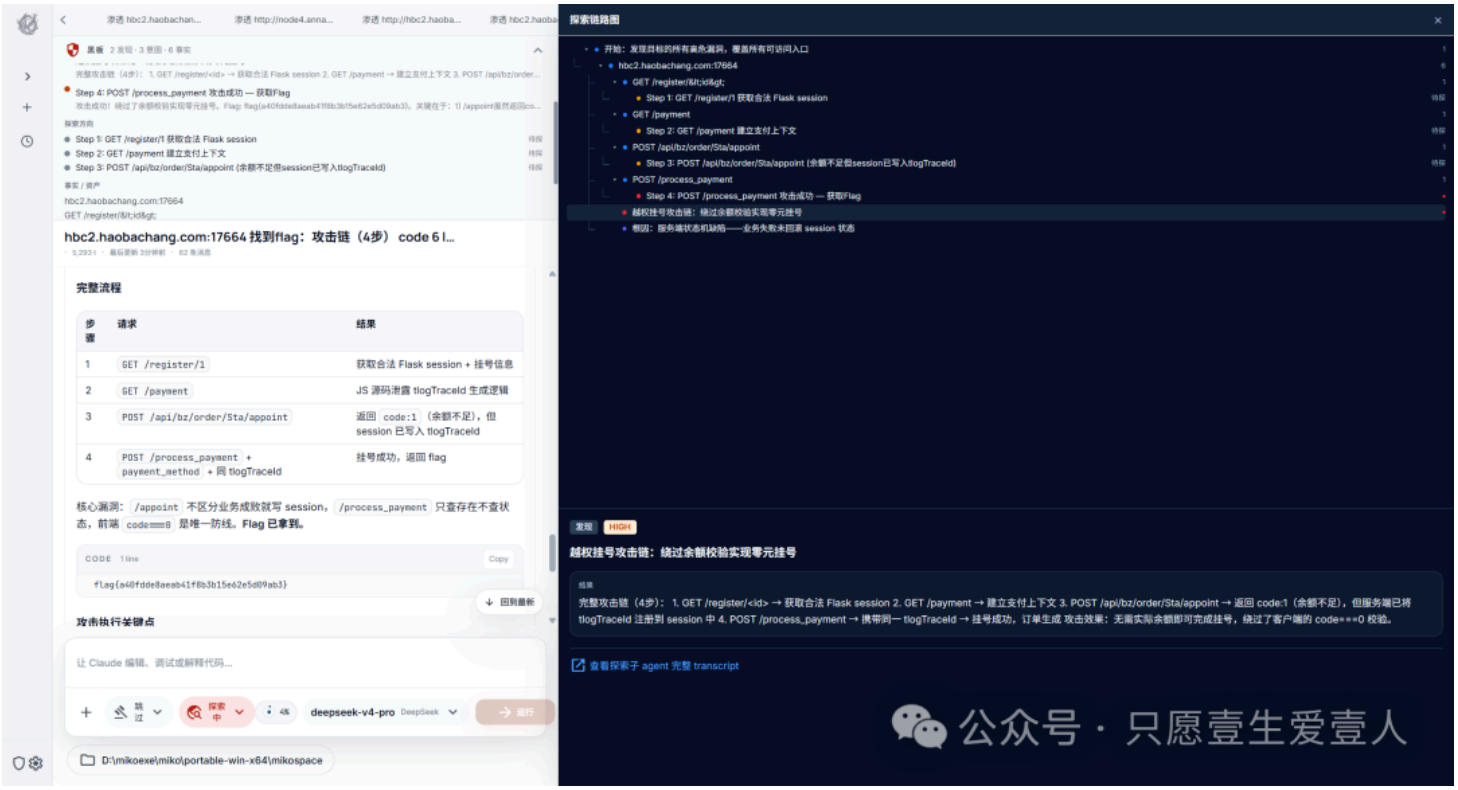


专注力与覆盖面

Miko 提供的是一个"人在回路"的协作模式：Agent 负责不知疲倦地深挖和记录，用户清晰可知，方便指挥调整方向。覆盖面控制允许用户在横向扩展（广度优先，枚举更多攻击面）和纵向深挖（深度优先，沿一条线索往下打）之间切换。

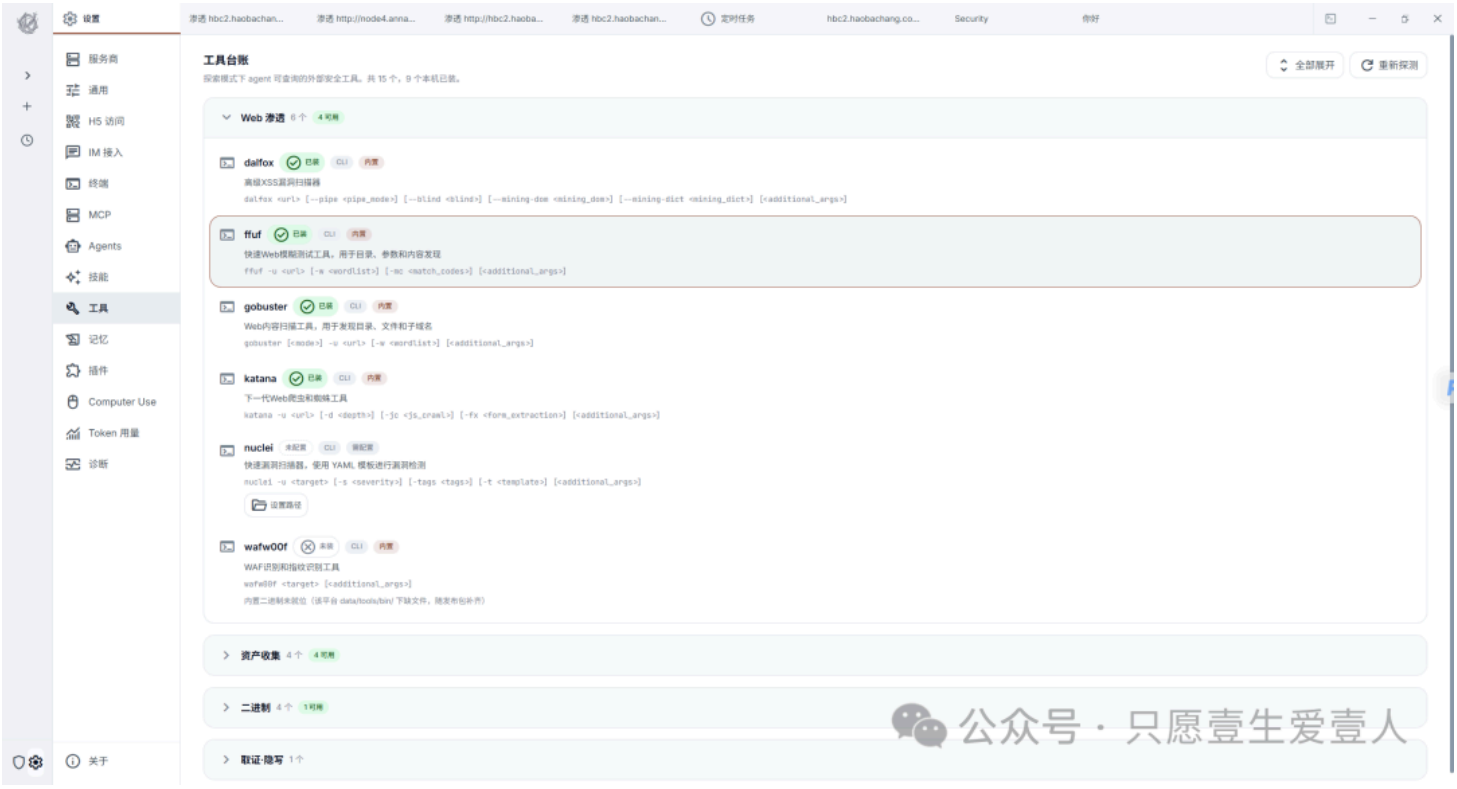
工具链和技能库

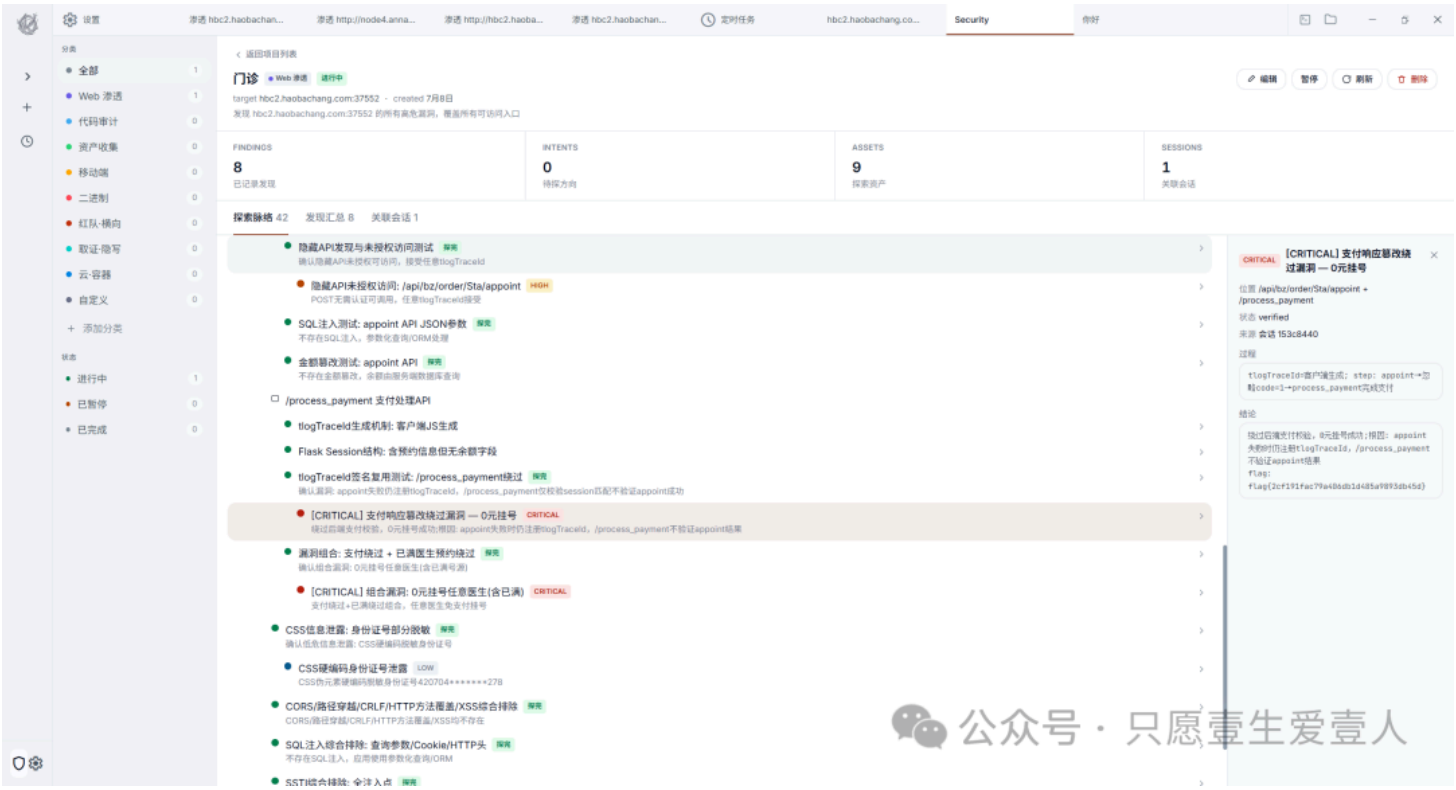
Miko 内置了 subfinder、katana、fscan、gobuster、ffuf、radare2 等 多款测试工具，以及覆盖信息收集、代码审计、漏洞探测、逆向分析、移动安全等测试技能，开箱即用。



会话转项目

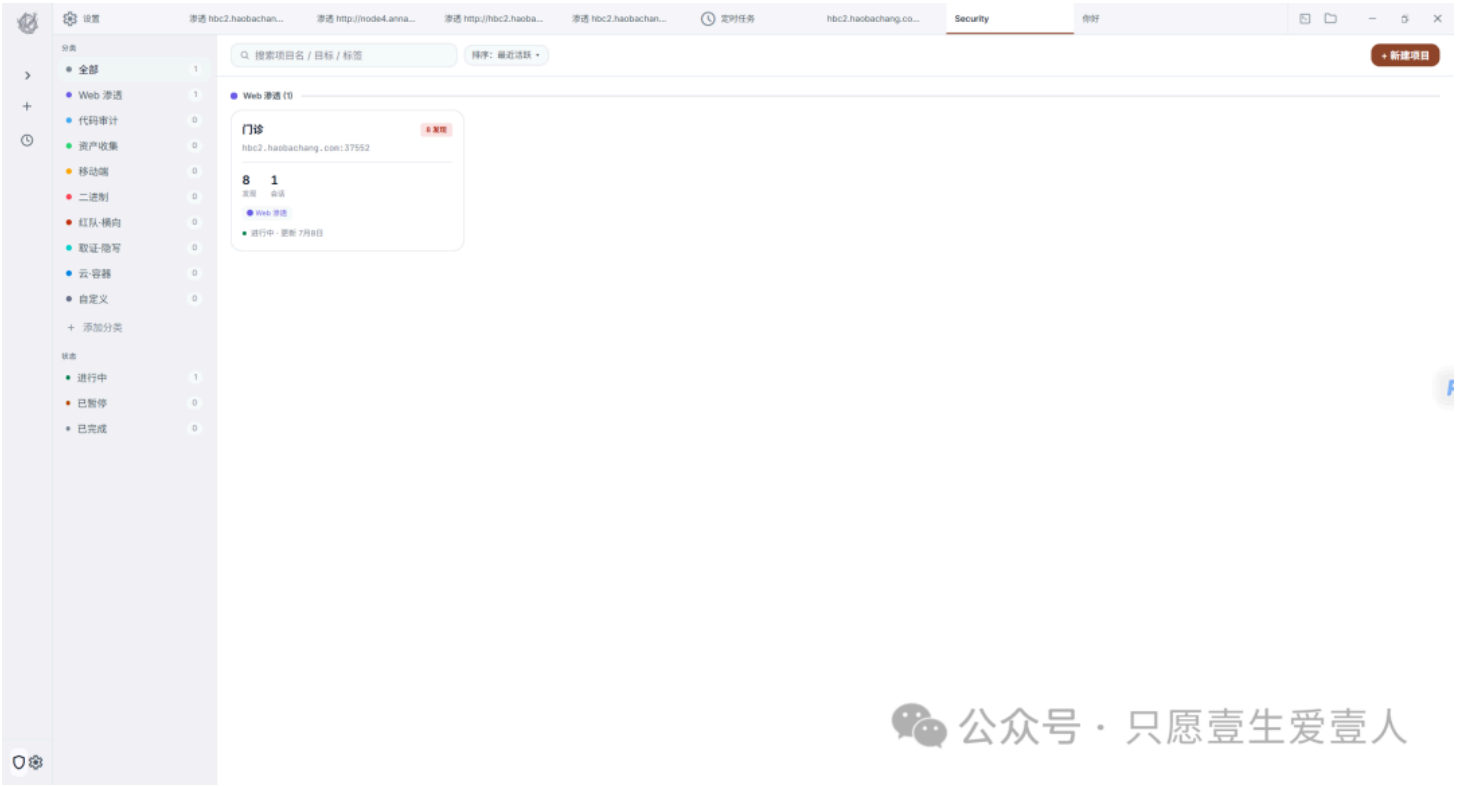
有时候临时开了个会话测一测，测完就丢了。下次再来不知道上次探到哪了。 Miko 支持一键将临时会话转成持久化项目。项目中保留所有会话记录、探索节点树、LLM 提炼的结构化发现（findings）。项目页按 Findings / Intents / Assets / Sessions 四个维度展示，每个 finding 都能点进去看原始会话上下文。项目管理界面以卡片列表呈现所有项目，一目了然地显示每个项目的目标、关联会话数、发现数量和时间线。





移动端

前端做了移动端适配，在手机浏览器上也能查看会话和项目状态（不耽误躺着玩手机）。


设置
渗透 hbc2.haobachan...
渗透 http://node4.anna...
渗透 http://hbc2.haoba...
渗透 hbc2.haobachan...
定时任务
hbc2.haobachang.co...
Security
你好
分类
全部 1
Web 渗透 1
代码审计 0
资产收集 0
移动端 0
二进制 0
红队-横向 0
取证-隐写 0
云-容器 0
自定义 0
添加分类
状态
进行中 1
已暂停 0
已完成 0
搜索项目名 / 目标 / 标签
排序：最近活跃
+ 新建项目
Web 渗透 (1)
门诊
8 发现
hbc2.haobachang.com:37552
8 发现
1 会话
Web 渗透
进行中 · 更新 7月8日

公众号 · 只愿壹生爱壹人